Mobile Development :

6 : Persistence and Storage for Mobile Development

Professor Imed Bouchrika
National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **Section 1 : Revision**
  - ***Null-Safety in Dart***
  - ***Asynchronous Programming in Dart***
  - ***Design Patterns : Singleton + Repository***
- **Section 2 : Storage Technology**
- **Section 3 : Shared Preferences**
- **Section 4 : Relational Data Storage**
  - ***Revision : Todo App without DB + Design Patterns***
  - ***SQFLITE***
  - ***Integration of Database***
- **Section 5 : Questions**

Section 1

Revision

Null Safety

Revision : Null-Safety for Dart

- **Final vs Const :**
  - **final :** The Value must be initialized during the declaration and will never change
  - **const :** The value will be computed during compile time and will never change later during execution.
  - Why use const : To optimize the performance of flutter via the reuse of the same constant widgets.

Revision : Null-Safety for Dart

- **? vs ?. vs ?? vs ! :**
  - ? : To say a variable can take the value Null during the declaration
    - **String? name ;**
  - ?. : to avoid null pointer exception and return null :
    - **return person?.name**
    - For the case when person is null, it returns null directly without accessing name
  - ?? : If the null, return the value on the right side:
    - return person??Person()
    - If person is null, instantiate a new Person and return it.

Revision : Null-Safety for Dart

- **? vs ?. vs ?? vs ! :**
  - ! : To cast away nullability (or simply get rid of the warnings)
    - return person!.name (Compiler will complain that person is nullable, we cast it to non-nullable.
  - **Warning : you use this only when you are completely certain that the object is non-nullable**

Async Coding

Writing Async Functions in Flutter

```
Future<String> getData() async{
    String data = await processLongRequest();
    print(data);
    return data
}
```

Writing Async Functions in Flutter

Any function that has a blocking code/ AWAIT, must be declared async + return Future

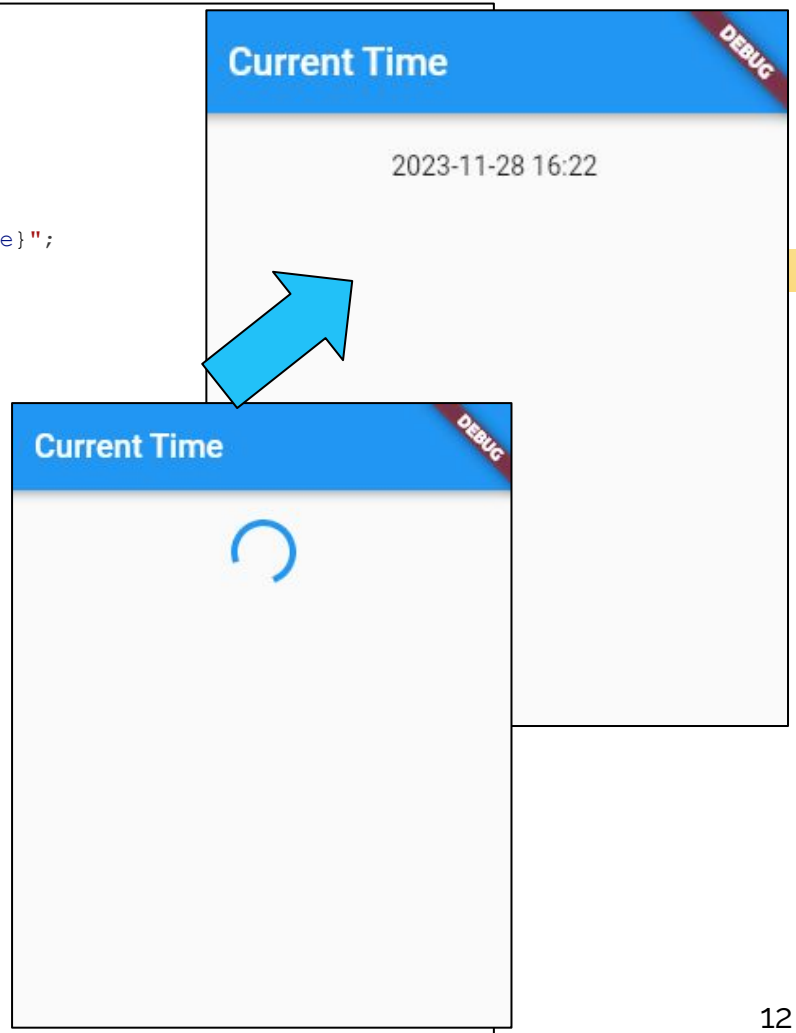
```
Future<String> getData() async{
    String data = await processLongRequest();
    print(data);
    return data
}
```

Writing Async Functions in Flutter

Await instructions must be only defined INSIDE A FUNCTION

```
Future<String> getData() async{
    String data = await processLongRequest();
    print(data);
    return data
}
```

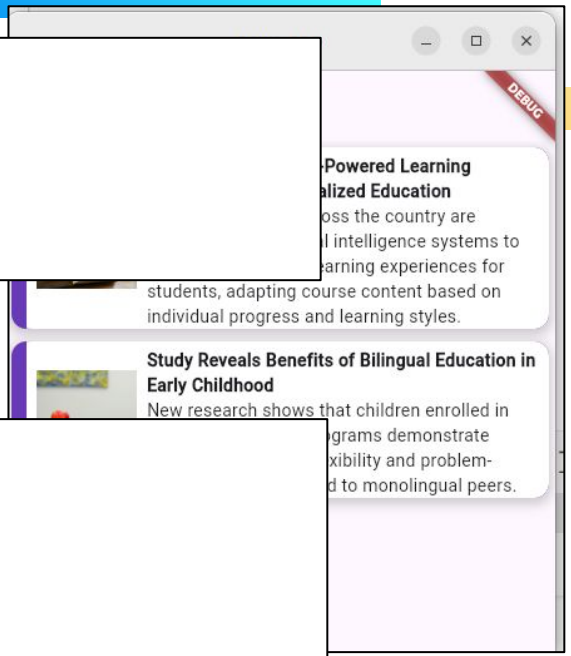
```
...

class _HomeScreenState extends State<HomeScreen> {
 Future<String> getCurrentTime() async {
   await Future.delayed(Duration(seconds: 5));
   DateTime time = DateTime.now();
   String strTime =
       "${time.year}-${time.month}-${time.day} ${time.hour}:${time.minute}";
   return strTime;
 }
 @override
 Widget build(BuildContext context) {
   Future<String> strTime = getCurrentTime();
   return Scaffold(
     appBar: AppBar(title: Text("Current Time")),
     body: Center(
         child: Column(
       children: [
         SizedBox(height: 20),
           FutureBuilder<String>(
               future: strTime,
               builder: (context, snapshot) {
                 if (snapshot.hasData) {
                   return Text(snapshot.data!);
                 } else if (snapshot.hasError) {
                   return Text("${snapshot.error}");
                 }

                 return CircularProgressIndicator();
               })
       ],
     )),
```



FutureBuilder Widget in Flutter

- **Simple S**
  - Do I need to load the news:
    1. Every time I refresh Widget UI?
    2. **V**

```
Widget build(BuildContext context) {
  news = NewsUtils.getNewsFromWeb();
  return Scaffold(
    appBar: AppBar(title: const Text('News App')),
    body: Center(
      child: FutureBuilder<List<Map>>(
```




```
@override
void initState() {
  super.initState();
  news = NewsUtils.getNewsFromWeb();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('News App')),
```

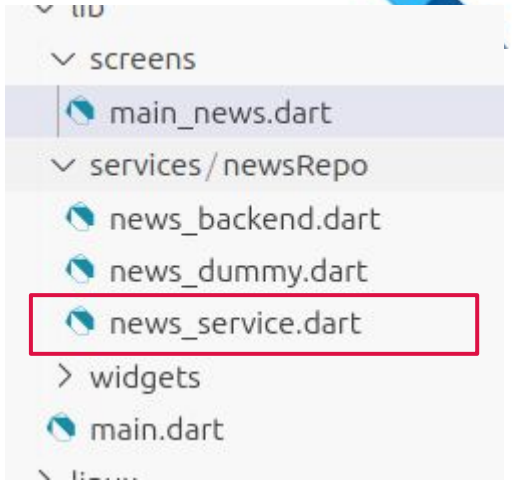
Design Patterns

Abstraction & Design Patterns for Structuring

- **Singleton Pattern**
  - A technique to ensure only one instance object is created
- **What's a design pattern:**
  - Technique or a solution to a common problem in programming.

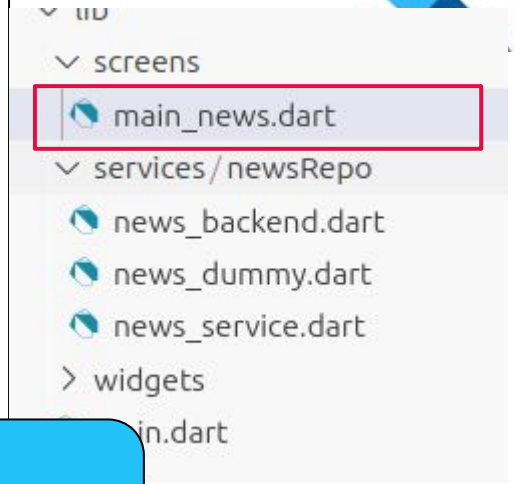
Abstraction & Design Pattern for Structuring

```dart
import 'package:news_app/services/newsRepo/news_dummy.dart';

abstract class AbstractNewsRepo {
 Future<List<Map>> getNewsData();

 static AbstractNewsRepo? _newsInstance;

 static AbstractNewsRepo getInstance() {
   _newsInstance ??= NewsDummy();
   return _newsInstance!; // For backend data
 }
}
```



Abstraction & Design Pattern for Structuring

```dart
class _NewsScreenState extends State<NewsScreen> {
 late Future<List<Map>> news;
 late AbstractNewsRepo newsRepo;
 @override
 void initState() {
   super.initState();
   newsRepo = AbstractNewsRepo.getInstance();
   news = newsRepo.getNewsData();
 }
```



Can we just simply say :

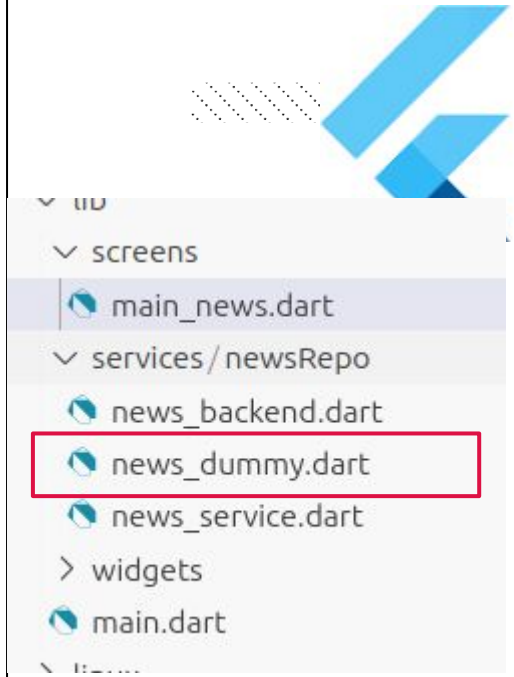
`newsRepo = new BackendNews();` **?**

In other files ? we may end up recreating more objects ?..

Abstraction & Design Patterns for Structuring

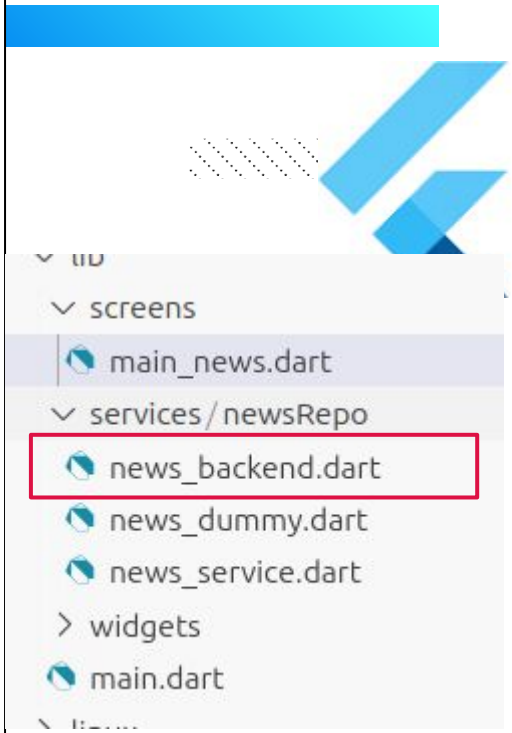
- **Repository Pattern**
  - A technique that acts as an abstraction layer between the business logic and the data access layer.
  - The business logic will not need to know about the data source.

```dart
class NewsDummy extends AbstractNewsRepo {
 @override
 Future<List<Map>> getNewsData() async {
   await Future.delayed(Duration(seconds: 5)); // Simulate network delay
   List<Map> news = [
     {
       "id": 1,
       "title":
           "Universities Adopt AI-Powered Learning Platforms for Personalized
Education",
       "image url":
           "https://images.unsplash.com/photo-1501504905252-473c47e087f8" ,
       "description":
           "Major universities across the country are implementing artificial
intelligence systems to create personalized learning experiences for students,
adapting course content based on individual progress and learning styles." ,
       "category_id": 3,
     },
     {
       "id": 2,
       "title":
           "Study Reveals Benefits of Bilingual Education in Early Childhood" ,
       "image url":
           "https://images.unsplash.com/photo-1503676260728-1c00da094a0b" ,
       "description":
           "New research shows that children enrolled in bilingual education
programs demonstrate enhanced cognitive flexibility and problem-solving skills
compared to monolingual peers." ,
       "category_id": 3,
     },
   ];
   return news;
 }
}
```



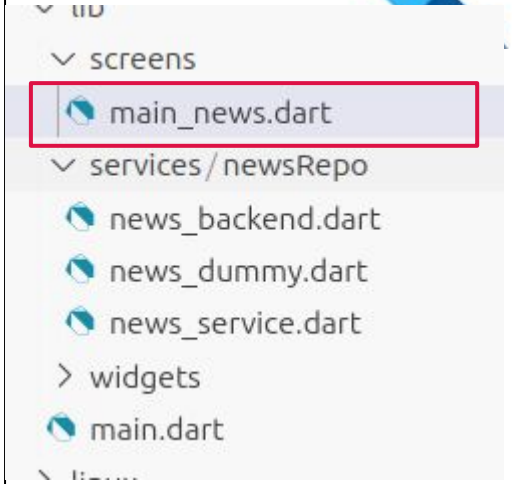
```dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:news_app/services/newsRepo/news_service.dart';

class NewsBackend extends AbstractNewsRepo {
 @override
 Future<List<Map>> getNewsData() async {
   print('Function : getNewsFromWeb : Fetching news from web...');
   try {
     final response = await http.get(
       Uri.parse(

'https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news.all.get ',
       ),
       headers: {'Content-Type': 'application/json'},
     );
     print('Response status: ${response.toString()}');
     if (response.statusCode == 200) {
       // Parse JSON response
       final List<dynamic> data = json.decode(response.body);
       return data.map((item) => item as Map<String, dynamic>).toList();
     } else {
       throw Exception('Failed to load news: ${response.statusCode}');
     }
   } catch (e) {
     throw Exception('Error fetching news: $e');
   }
 }
}
```

flutter pub add http



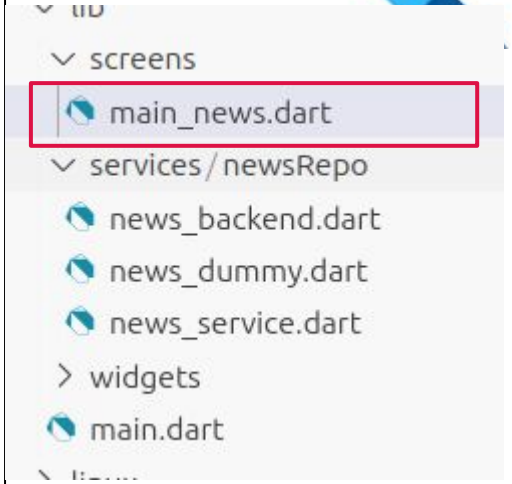
Abstraction & Design Pattern for Structuring

```dart
class _NewsScreenState extends State<NewsScreen> {
 late Future<List<Map>> news;
 late AbstractNewsRepo newsRepo;
 @override
 void initState() {
   super.initState();
   newsRepo = AbstractNewsRepo.getInstance();
   news = newsRepo.getNewsData();
 }
```



Abstraction & Design Pattern for Structuring

```dart
class _NewsScreenState extends State<NewsScreen> {
 late Future<List<Map>> news;
 late AbstractNewsRepo newsRepo;
 @override
 void initState() {
   super.initState();
   newsRepo = AbstractNewsRepo.getInstance();
   news = newsRepo.getNewsData();
 }
```



Is the news loaded from the Web ? Dummy data ? from a local DB ? from a Cache ?

This is not the concern of the this screen

Section 2

Persistence Technologies

Data Persistence using Flutter

- **Data Persistence/Storage**
  - **Do we need to store data ?**
    - User's preferences ? theme ? language ? settings..
    - User data :
      - Depending on the App (Notes, Favorite Items, Photos…)
  - **Where to store data ?**

Data Persistence using Flutter

- **Data Persistence/Storage**
  - Data can be stored for mobile apps using :
    - Shared Preferences
    - Local Databases
    - As Files in the filesystem
    - Cloud Services :
      - Firestore (By Google Firebase services)
      - AWS + ...

Data Persistence using Flutter

- **Data Persistence/Storage**
  - Data can be stored for mobile apps using :
    - Shared Preferences : Simple data stored locally.
    - Local Databases : usually relational data for **offline usage**
    - As Files in the filesystem : for binary files (including assets)
    - Cloud Services : When there is a need to sync
      - Firestore (By Google Firebase services)
      - AWS + ...

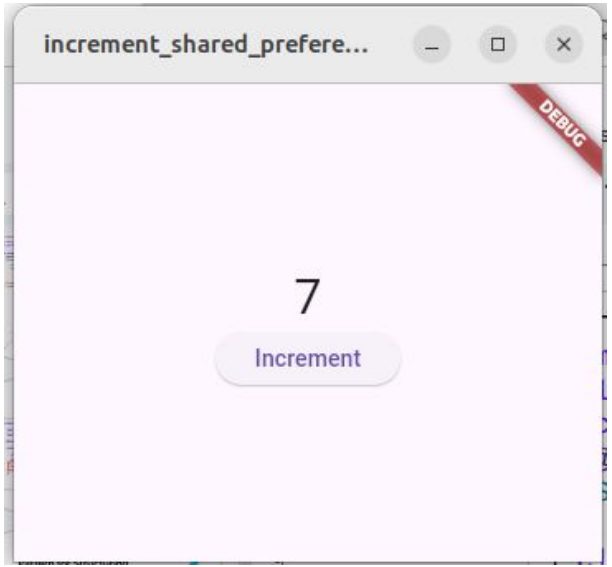
Data Persistence using Flutter

- **Data Persistence/Storage**
  - **Data Structuring and Architecture :**
    - Simple key:value ⇒ Shared Preferences
    - Relational : When you need to explore the relationship between different entities + many low level operations per row.
    - NoSQL: less structuring
      - Collection/Documents-based
      - Key:Value data
    - Hybrid : Relational with NoSQL inside columns

Section 3

SharedPreferences

```dart
import 'package:flutter/material.dart' ;
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() => _IncrementState();
}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Column(
           mainAxisAlignment : MainAxisAlignment.center,
           children: [
             Text('$increment_value', style: TextStyle(fontSize: 30)),
             ElevatedButton(
               onPressed: incrementValue,
               child: const Text('Increment'),
             ),
           ],
         ),
       ),
     ),
   );
 }
 bool incrementValue() {
   increment_value++;
   setState(() {});
   return true;
 }
}
```



Data Persistence using Flutter

- **Shared Preferences in Flutter**
  - It is a way to store primitive data in the form **key:value** using the class ***SharedPreferences***
  - It is recommended to use it for small data
  - Available for Mobile, Web and Desktop Apps
  - https://pub.dev/packages/shared_preferences
  - Hive is a similar popular library
    - https://pub.dev/packages/hive

Data Persistence using Flutter

- **Shared Preferences in Flutter**

  1. To Add the Dependency :

     flutter pub add shared_preferences

  2. Load the sharedPreference instance

```
final SharedPreferences shared_prefs = await SharedPreferences.getInstance();
```

Data Persistence using Flutter

Which design Pattern ?

- **Shared Preferences in Flutter**
  1. To Add the Dependency :

     flutter pub add shared_preferences

  2. Load the sharedPreference instance

```
final SharedPreferences shared_prefs = await SharedPreferences.getInstance();
```

Data Persistence using Flutter

Where to place this line of code ?

+

It has await ?

- **Shared Preferences in Flutter**
  1. To Add the Dependency :

     flutter pub add shared_preferences

  2. Load the sharedPreference instance

```
final SharedPreferences prefs = await SharedPreferences.getInstance();
```

```dart
import 'package:flutter/material.dart' ;
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() => _IncrementState();
}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Colum
           mainAxisAl
           children:
             Text('$i
             Elevated
               onPres
               child:
           ),
         ],
       ),
     ),
   ),
   );
 }
 bool incrementValue(
   increment value++;
   setState(() {});
   return true;
 }
}
```

Import libraries + define shared_ref Instance

01

```dart
import 'package:flutter/material.dart' ;
import 'package:shared_preferences/shared_preferences.dart';

class Increment extends StatefulWidget {
 const Increment({super.key});

 @override
 State<Increment> createState() => _IncrementState();
}

class _IncrementState extends State<Increment> {
 var increment_value = 0;

 late final SharedPreferences shared_prefs;
```

```dart
import 'package:flutter/material.dart' ;
class Increment extends StatefulWidget {
 const Increment ({super.key});
 @override
 State<Increment> createState () => _IncrementState ();
}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Colum
           mainAxisAl
           children:
             Text('$i
             Elevated
               onPres
               child:
           ),
         ],
       ),
     ),
   ),
   );
 }
 bool incrementValue (
   increment value ++;
   setState (() {});
   return true;
 }
}
```

Initialize + load data

02

```dart
class _IncrementState extends State<Increment> {
 var increment_value = 0;

 late final SharedPreferences shared_prefs;
 @override
 void initState() {
   super.initState();
   initCounter();
 }
 Future<bool> initCounter() async {
   shared_prefs = await SharedPreferences.getInstance();
   increment_value = shared_prefs.getInt('increment_value') ?? 0;
   return true;
 }
```

Data Persistence using Flutter

- **Shared Preferences in Flutter**

3. Write Data

```
await prefs.setInt('counter', 10);
await prefs.setBool('repeat', true);
await prefs.setDouble('decimal', 1.5);
await prefs.setString('action', 'Start');
await prefs.setStringList('items', <String>['Earth', 'Moon', 'Sun']);
```

Data Persistence using Flutter

How to save Map Object to SharedPreferences ?

- **Shared Preferences in Flutter**

3. Write Data

```
await prefs.setInt('counter', 10);
await prefs.setBool('repeat', true);
await prefs.setDouble('decimal', 1.5);
await prefs.setString('action', 'Start');
await prefs.setStringList('items', <String>['Earth', 'Moon', 'Sun']);
```

```dart
import 'package:flutter/material.dart' ;
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() => _IncrementState();
}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Column(
           mainAxisAlignment: Mai
           children: [
             Text('$increment_val
             ElevatedButton(
               onPressed: increme
               child: const Text
             ),
           ],
         ),
       ),
     ),
   );
 }
 bool incrementValue() {
   increment_value++;
   setState(() {});
   return true;
 }
}
```

Initialize + load data

03

```dart
bool incrementValue() {
  increment_value++;
  shared_prefs.setInt('increment_value', increment_value);
  setState(() {});
  return true;
}
```

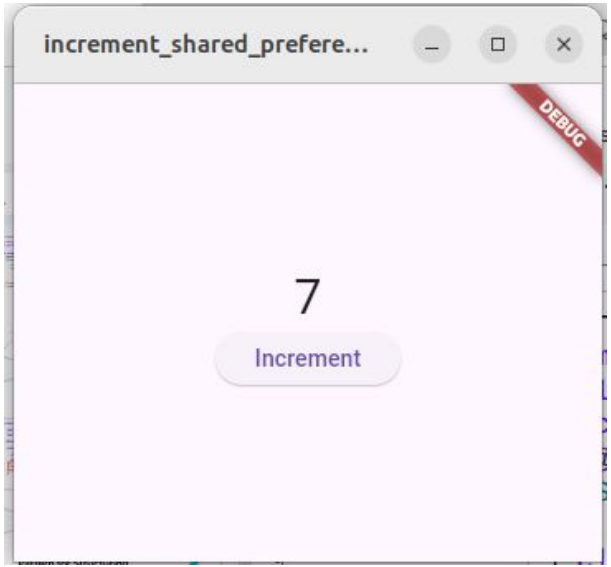
Data Persistence using Flutter

- **Shared Preferences in Flutter**

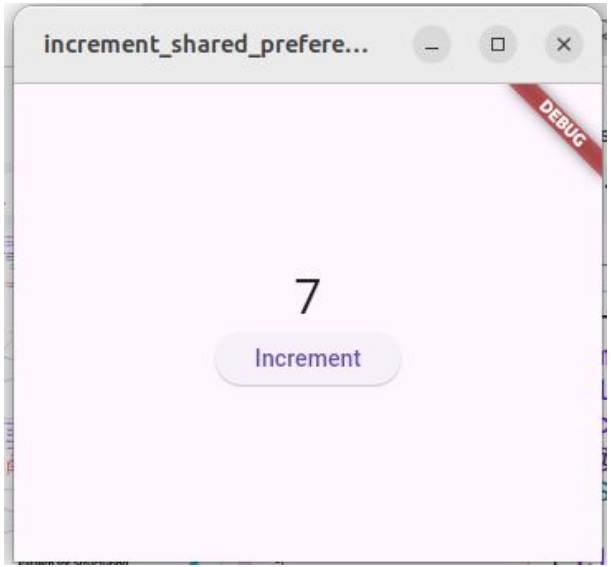
4. Read Data

```
final int? counter = prefs.getInt('counter');
final bool? repeat = prefs.getBool('repeat');
final double? decimal = prefs.getDouble('decimal');
final String? action = prefs.getString('action');
final List<String>? items = prefs.getStringList('items');
```

```dart
import 'package:flutter/material.dart' ;
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() => _IncrementState();
}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Column(
           mainAxisAlignment: MainAxisAlignment.center,
           children: [
             Text('$increment_value', style: TextStyle(fontSize: 30)),
             ElevatedButton(
               onPressed: incrementValue,
               child: const Text('Increment'),
             ),
           ],
         ),
       ),
     ),
   );
 }
 bool incrementValue() {
   increment_value++;
   setState(() {});
   return true;
 }
}
```

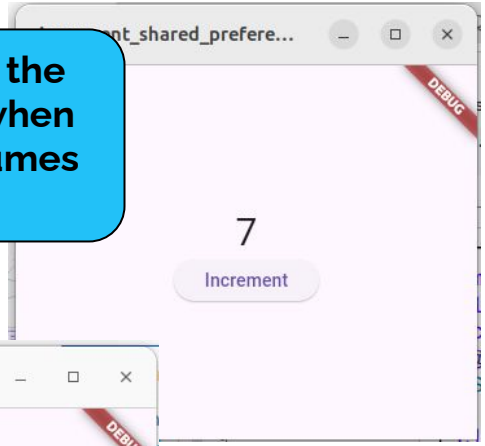


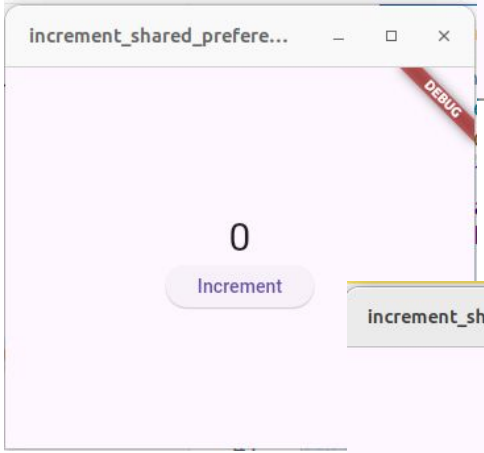
```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() =>  IncrementState();}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 late final SharedPreferences shared_prefs;
 @override
 void initState() {
   super.initState();
   initCounter();
 }
 Future<bool> initCounter() async {
   shared_prefs = await SharedPreferences.getInstance();
   increment_value = shared_prefs.getInt('increment_value') ?? 0;
   return true;
 }
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Column(
           mainAxisAlignment: MainAxisAlignment.center,
           children: [
             Text('$increment_value', style: TextStyle(fontSize: 30)),
             ElevatedButton(
               onPressed: incrementValue,
               child: const Text('Increment'),),],),),),  );
 }
 bool incrementValue() {
   increment_value++;
   shared_prefs.setInt('increment_value', increment_value);
   setState(() {});
   return true;
 }
}
```

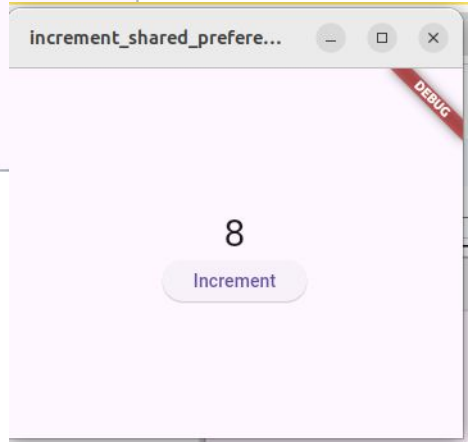


```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() =>  IncrementState();}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 late final SharedPreferences shared_prefs;
 @override
 void initState() {
   super.initState();
   initCounter();
 }
 Future<bool> initCounter() async {
   shared_prefs = await SharedPreferences.getInstance();
   increment_value = shared_prefs.getInt('increment_value') ?? 0;
   return true;
 }
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Column(
           mainAxisAlignment: MainAxisAlignment.center,
           children: [
             Text('$increment_value', style: TextStyle(fontSize: 30)),
             ElevatedButton(
               onPressed: incrementValue,
               child: const Text('Increment'),),],),),),  );
 }
 bool incrementValue() {
   increment_value++;
   shared_prefs.setInt('increment_value', increment_value);
   setState(() {});
   return true;
 }
}
```

Problem : When opening the App, it shows ZERO, but when clicking increment, it resumes fine ?

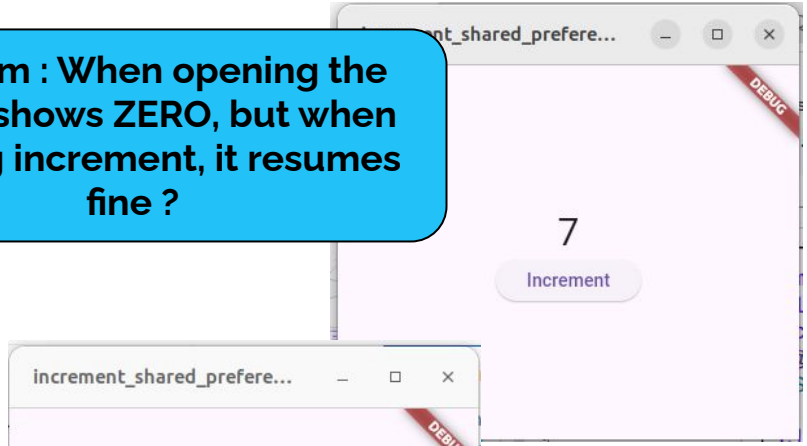


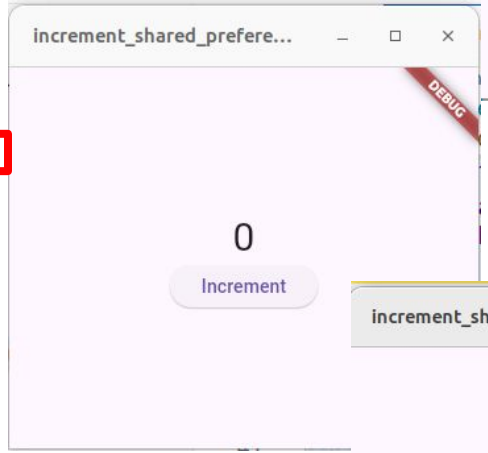


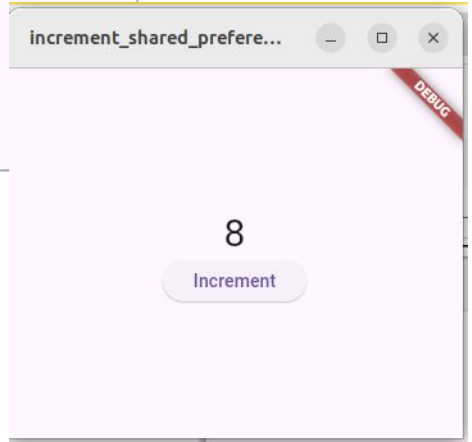


```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
class Increment extends StatefulWidget {
 const Increment({super.key});
 @override
 State<Increment> createState() =>  IncrementState();}
class  IncrementState extends State<Increment> {
 var increment_value = 0;
 late final SharedPreferences shared_prefs;
 @override
 void initState() {
   super.initState();
   initCounter();
 }
 Future<bool> initCounter() async {
   shared_prefs = await SharedPreferences.getInstance();
   increment_value = shared_prefs.getInt('increment_value') ?? 0;
   setState(() {});
   return true;
 }
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Container(
       child: Center(
         child: Column(
           mainAxisAlignment: MainAxisAlignment.center,
           children: [
             Text('$increment_value', style: TextStyle(fontSize: 30)),
             ElevatedButton(
               onPressed: incrementValue,
               child: const Text('Increment'),),],),),),  );
 }
 bool incrementValue() {
   increment_value++;
   shared_prefs.setInt('increment_value', increment_value);
   setState(() {});
   return true;
 }
```

Problem : When opening the App, it shows ZERO, but when clicking increment, it resumes fine ?



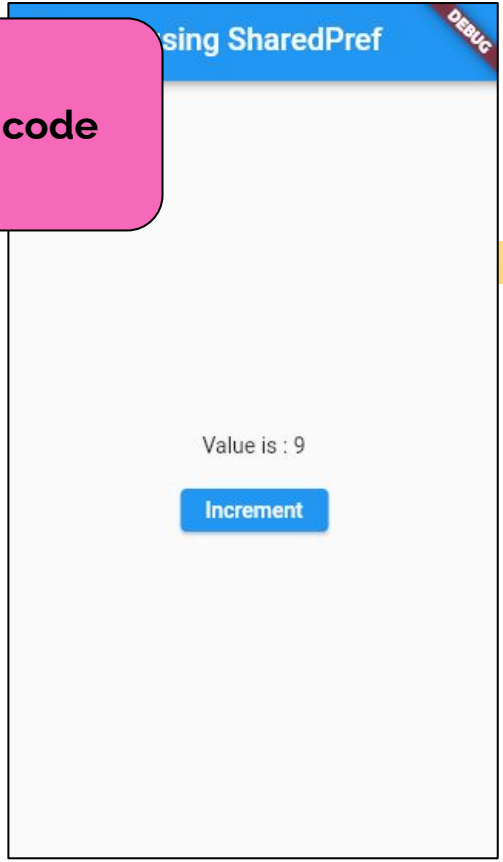




Data Persistence Flutter

Never mix BL with UI code

```
@override
Widget build(BuildContext context) {
  return Scaffold(
      appBar: AppBar(title: Text("Counter using SharedPref ")),
      body: Center(
          child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        crossAxisAlignment: CrossAxisAlignment.center,
        children: [
          Text("Value is : ${increment}"),
          SizedBox(height: 20),
          ElevatedButton(

              onPressed: () async {
                increment = increment + 1;
                await prefs.setInt("incrementNumber", increment);
                setState(() {});
              },

              child: Text("Increment"))
        ],
      )));
}
```



Data Persistence using Flutter

- **Shared Preferences in Flutter**

  5. Remove Data

```
// Remove data for the 'counter' key.

await prefs.remove('counter');
```

Data Persistence using Flutter

- **Shared Preferences in Flutter**
  - How frequent to save :
    - Recommended After each modification of a variable
    - On closing the App ? depending how critical is the data

Section 4

Offline Relational Data Storage

Persistence of Data for Mobile Apps

- **Todo App**
  - Create a Todo App where a user can add task and mark them as completed.
  - The app need to be created from one screen.
  - The data for Todo will be saved as a local variable, no need for persistent storage.
  - User can delete an item

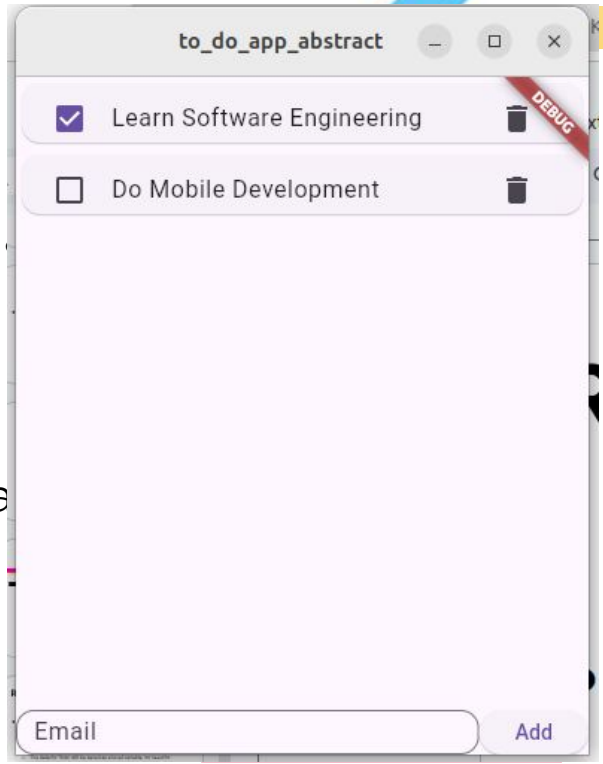
Todo Without Database

Revision : Todo App using FutureBuilder

- **Todo App**
  - Create a Todo App where a user can add task and mark them as completed.
  - The app need to be created from one screen.
  - The data for Todo will be saved as a local variable, no need for persistent storage.
  - User can delete an item

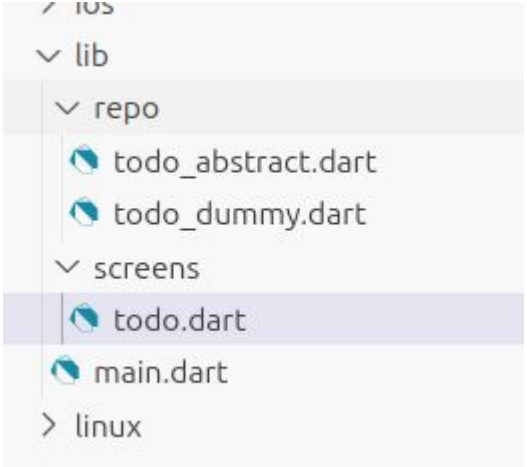
Revision : Todo App using FutureBuilder

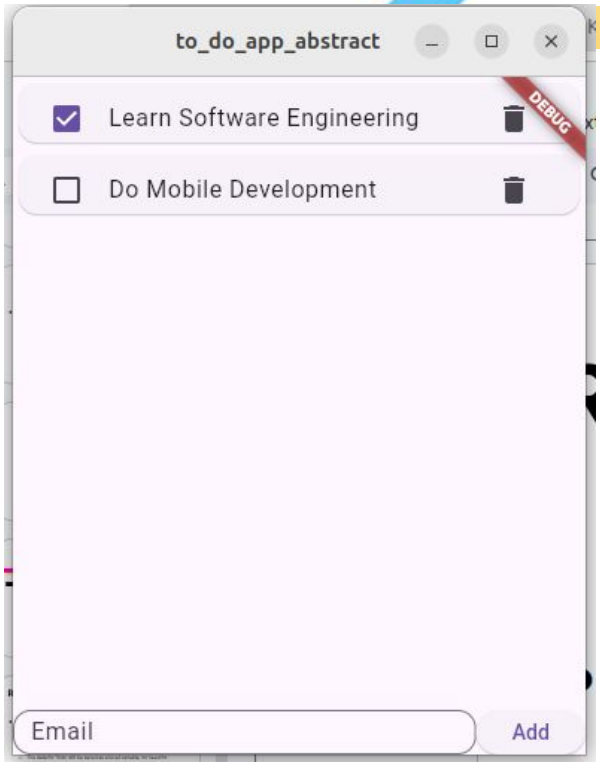
- **Todo App**
  - Create a Todo App where a user can add task
    completed.
  - The app need to be created from one screen.
  - The data for Todo will be saved as a local varia
    storage.
  - User can delete an item



Revision : Todo App using FutureBuilder

- **Todo App**
  - Project Structure





Revision : Todo App using FutureBuil

- **Todo App**
  - Main Screen

```dart
import 'package:flutter/material.dart';
import 'package:to_do_app_abstract/screens/todo.dart';

void main() {
 runApp(const MainApp());
}

class MainApp extends StatelessWidget {
 const MainApp({super.key});

 @override
 Widget build(BuildContext context) {
   return const MaterialApp(home: TodoHomeScreen());
 }
}
```

Revision : Todo App using FutureBuilder

- **Todo App**
  - Abstraction for loading Data

```dart
import 'todo_dummy.dart';

abstract class AbstractTodoRepo {
 Future<List<Map>> getData();
 Future<bool> insertItem(Map<String, dynamic> item);
 Future<bool> deleteItem(int index);
 Future<bool> updateItem(int index, Map<String, dynamic> item);

 static AbstractTodoRepo? _todoInstance;

 static AbstractTodoRepo getInstance() {
   _todoInstance ??= TodoDummy();
   return _todoInstance!; // For backend data
 }
}
```

Revision : To Future

- **Todo App**
  - Abstraction for Data

```dart
import 'package:to_do_app_abstract/repo/todo_abstract.dart';
class TodoDummy extends AbstractTodoRepo {
 List<Map<String, dynamic>> data = [
   {'title': 'Learn Software Engineering', 'done': true},
   {'title': 'Do Mobile Development', 'done': false},
 ];
 @override
 Future<List<Map>> getData() async {
   await Future.delayed(Duration(seconds: 5)); // Simulate network delay
   return data;
 }
 @override
 Future<bool> deleteItem(int index) async {
   data.removeAt(index);
   return true;
 }
 @override
 Future<bool> insertItem(Map<String, dynamic> item) async {
   data.add(item);
   return true;
 }
 @override
 Future<bool> updateItem(int index, Map<String, dynamic> item) async {
   data[index] = item;
   return true;
 }
}
```

Revision : To
Future

- **Todo App**
  - Abstraction for Data

```dart
import 'package:flutter/material.dart';
import 'package:to_do_app_abstract/repo/todo_abstract.dart';

class TodoHomeScreen extends StatefulWidget {
 const TodoHomeScreen({super.key});
 @override
 State<TodoHomeScreen> createState() => _TodoHomeScreenState();
}

class _TodoHomeScreenState extends State<TodoHomeScreen> {
 final TextEditingController _controller = TextEditingController();

 late Future<List<Map>> data;
 late AbstractTodoRepo repo;

 @override
 void initState() {
   super.initState();
   repo = AbstractTodoRepo.getInstance();
 }
```

Revision : To
Future

- **Todo App**
  - Abstraction for Data

```
@override
Widget build(BuildContext context) {
  data = repo.getData();
  return Scaffold(
    body: Column(
      children: [
        Expanded(
          child: FutureBuilder(
            future: data,
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Center(child: CircularProgressIndicator());
              }
              if (snapshot.hasData && snapshot.data != null) {
                List<Map<String, dynamic>> tmp_data =
                    snapshot.data! as List<Map<String, dynamic>>;
                return ListView.builder(
                  itemCount: tmp_data.length,
                  itemBuilder: (context, index) {
                    return getItemTodo(index, tmp_data[index]);
                  },
                );
              } else if (snapshot.hasError) {
                return Center(child: Text('Error: ${snapshot.error}'));
              }
              return const Center(child: Text('No Data Available'));
            },
          ),
        ),
        getBottomForm(),
      ],
```

Revision : To Future

- **Todo App**
  - Abstraction for Data

```
Widget getBottomForm() {
  return SizedBox(
    height: 30,
    child: Row(
      children: [
        Expanded(
          child: TextField(
            controller: _controller,
            decoration: InputDecoration(
              labelText: 'Email',
              border: OutlineInputBorder(
                borderRadius: BorderRadius.circular(12),
              ),
            ),
          ),
        ),
        ElevatedButton(
          onPressed: () async {
            await repo.insertItem({
                'title': _controller.text, 'done': false});
            _controller.text = '';
            setState(() {});
          },
          child: Text('Add'),
        ),
      ],
    ),
  );
}

Widget getItemTodo(int index, Map<String, dynamic> item) {
```

Revision : To Future

- **Todo App**
  - Abstraction for Data

```
Widget getItemTodo(int index, Map<String, dynamic> item) {
  return Card(
    child: Container(
      height: 40,
      child: ListTile(
        title: Text(item['title']),
        leading: Checkbox(value: item['done'], onChanged: (value) {}),
        trailing: IconButton(
          icon: const Icon(Icons.delete),
          onPressed: () async {
            await repo.deleteItem(index);
            setState(() {});
          },
        ),
      ),
    ),
  );
 }
}
```

SQLite For Mobile

Relational Databases : SQLite

- **Relational Databases : SQLite**
  - SQLite is a well-regarded SQL-based relational database management system (RDBMS). It is
    - Open source
    - Standards-compliant, implementing most of the SQL standard
    - Lightweight
    - Single-tier
    - ACID compliant

Relational Databases : SQLite

- **Relational Databases : SQLite:**
  - SQLite is implemented as a compact C library that's included as part of the Android software stack
  - Each SQLite database is an integrated part of the application that created it. This reduces external dependencies, minimizes latency, and simplifies transaction locking and synchronization.

Data Persistence

Relational Embedded Databases

- **SQL Reminder : Creating Tables**

```
CREATE TABLE IF NOT EXISTS  expenses  (
   expense_id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT  NOT NULL,
   price REAL NOT NULL,
   date REAL NOT NULL,
   image BLOB NULL
);
```

Data Persistence

Relational Embedded Databases

- **SQL Reminder : Creating Tables**
  - Data Types :
    - VARCHAR(N)
    - TEXT
    - INT
    - LONG
    - DATE
    - ENUM ..

Data Persistence

Relational Embedded Databases

- **SQL Reminder : Searching and Retrieving Data**

```
SELECT table_name.column1,...FROM table_name WHERE table_name.column1>1
```

```
SELECT table_name.column1,...FROM table_name , table_two WHERE
        table_name.foreign_id=table_two.id AND
         table_name.column1>1
```

```
SELECT table_name.column1,...FROM table_name
                LEFT JOIN table_two ON table_name.foreign_id=table_two.id
        WHERE  table_name.column1>1  ORDER BY table_name.column DESC  LIMIT 10
```

Data Persistence

Relational Embedded Databases

- **SQL Reminder : Updating Data**

```
UPDATE table_name SET
        column_name1='VALUE',
        column_name2='another VALUE',
WHERE
        column_name5='some value'
```

Data Persistence using Flutter

01

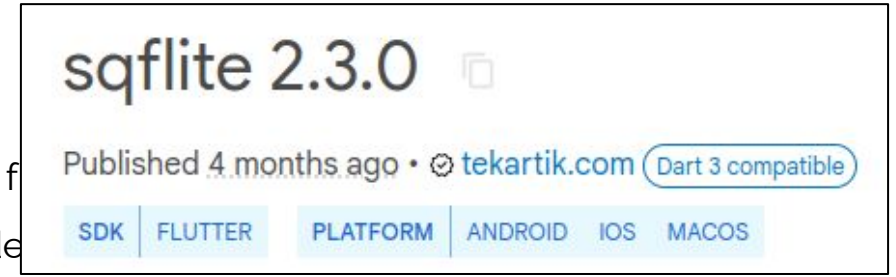
- **Steps to started : Plugins & Packages**
  - SQFLite is the flutter plugin for using SQLite
    - https://pub.dev/packages/sqflite
  - To get started install:
    - sqflite : package provides classes and functions to interact with a SQLite database.
    - path : package provides functions to define the location for storing the database on disk.
      - **flutter pub add sqflite path**
  - Import the packages:

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
```

Data Persistence using Flutter

01

- **Steps to started : Plugins & Packages**
  - SQFLite is the flutter plugin for using SQLite
    - https://pub.dev/packages/sqflite
  - To get started install:
    - sqflite : package provides classes and f
    - path : package provides functions to de
      - **flutter pub add sqflite path**
  - Import the packages:




```dart
import 'dart:async';

import 'package:flutter/widgets.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
```

Data Persistence using Flutter

01

- **Steps to started : Plugins & Packages**
  - Running App on **Windows and Linux** :
    1. Add the following library :

flutter pub add sqflite_common_ffi

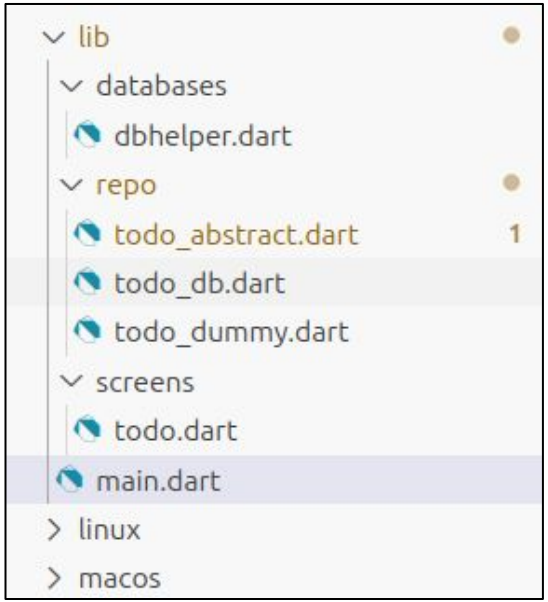
    2. Initialize the library :

```dart
import 'package:flutter/material.dart';
import 'package:to_do_app_abstract/screens/todo.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'dart:io' show Platform;
void main() {
 if (Platform.isLinux || Platform.isWindows) {
   sqfliteFfiInit();
   databaseFactory = databaseFactoryFfi;
 }
 runApp(const MainApp());
}
```

Data Persistence using Flutter

02

- **Steps to started : DBHelper Class**
  - Create a folder **databases** inside **lib**
  - Create **dbhelper.dart** file :
    - Shall contain :
      - SQL Database name
      - SQL Database version
      - SQL Code for creating/upgrading the tables.
      - Singleton Method to get an instance of the database



```dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class DBHelper {
 static const  database_name = "ENSIA_MY_DB.db";
 static const  database_version = 4;
 static var database;

 static Future getDatabase() async {
   if (database != null) {
     return database;
   }
   database = openDatabase(
     join(await getDatabasesPath(), _database_name),
     onCreate: (database, version) {
       database.execute('''
         CREATE TABLE  todo (
             id INTEGER PRIMARY KEY AUTOINCREMENT,
             title TEXT,
             done INTEGER,
             duedate TEXT,
             create_date TEXT)
       ''');
     },
     version:  database_version,
     onUpgrade: (db, oldVersion, newVersion) {      },
   );
   return database;
 }
}
```

02

Data Persistence using Flutter

03

- **Steps to started : Access the databases**
  - Create a class for each table when needed and **Either**
    1. Define static methods within each class :
       - To use within UI : **CLASS_NAME.method_NAME**
         - TODO_DB.get_data() **(Coupling UI with DB Tech)**
    2. Recommended, use the Repo design pattern, create a class for each database table + implement the abstract methods.
       - To use : **repo.getData()**

```dart
import 'package:sqflite/sqflite.dart';
import 'databases/dbhelper.dart';
class TodoDB {
 static Future<List<Map<String, dynamic>>> getAllToDos() async {
   final database = await DBHelper.getDatabase();
   return database.rawQuery('''SELECT
           todo.id ,
           todo.title,
           todo.done
         from todo
         ''');
 }
 static Future insertToDo(Map<String, dynamic> data) async {
   final database = await DBHelper.getDatabase();
   database.insert("todo", data, conflictAlgorithm: ConflictAlgorithm.replace);
 }
 static void deleteToDo(int id) async {
   final database = await DBHelper.getDatabase();
   database.rawQuery("""delete from  todo where id=?""", [id]);
 }
 static void setDone(int id, bool flag) async {
   final database = await DBHelper.getDatabase();
   int value = flag ? 1 : 0;
   database.rawQuery("""update  todo set done=? where id=?""", [value, id]);
 }
}
```

03

Class for each table with Static methods
Not recommended

```dart
import 'package:to_do_app_abstract/repo/todo_abstract.dart';
class TodoDummy extends AbstractTodoRepo {
 List<Map<String, dynamic>> data = [
   {'title': 'Learn Software Engineering', 'done': true},
   {'title': 'Do Mobile Development', 'done': false},
 ];
 @override
 Future<List<Map>> getData() async {
   await Future.delayed(Duration(seconds: 5)); // Simulate network delay
   return data;
 }
 @override
 Future<bool> deleteItem(int index) async {
   data.removeAt(index);
   return true;
 }
 @override
 Future<bool> insertItem(Map<String, dynamic> item) async {
   data.add(item);
   return true;
 }
 @override
 Future<bool> updateItem(int index, Map<String, dynamic> item) async {
   data[index] = item;
   return true;
 }
}
```

03

Todo Dummy ?

```dart
import 'package:sqflite/sqflite.dart' ;
import 'package:to do app abstract/repo/todo_abstract.dart'  ;
import '../databases/dbhelper.dart' ;
class TodoDB extends AbstractTodoRepo {
 @override
 Future<List<Map>> getData() async {
   final database = await DBHelper.getDatabase();
   return database.rawQuery('''SELECT
          todo.id ,
          todo.title,
          todo.done
        FROM todo
        ''');
 }
 @override
 Future<bool> deleteItem(int index) async {
   final database = await DBHelper.getDatabase();
   database.rawQuery("""delete from  todo where id=?""", [index]);
   return true;
 }
 @override
 Future<bool> insertItem(Map<String, dynamic> item) async {
   final database = await DBHelper.getDatabase();
   database.insert("todo", item, conflictAlgorithm: ConflictAlgorithm.replace);
   return true;
 }
 @override
 Future<bool> updateItem(int index, Map<String, dynamic> item) async {
   final database = await DBHelper.getDatabase();
   database.update(index, item);
   return true;
 }
```

03

Todo DB ?

Data Persistence using Flutter

04

- **Steps to started : UI Integration**
  - The case of static methods

You have to go through all UI elements when needed and edit

```
class _HomeScreenState extends State<HomeScreen> {
 late Future<List<Map>> data;

 String _tx_title_value = '';
 final _tx_title_controller = TextEditingController();

 @override
 Widget build(BuildContext context) {
   data = TodoDB.getAllToDos();
   return Scaffold(
```

Data Persistence using Flutter

04

- **Steps to started : UI Integration**
  - The case of Repo Design Pattern

UI code will not be changed

```dart
import 'package:to do app_abstract/repo/todo_db.dart';
import 'todo dummy.dart';
abstract class AbstractTodoRepo {
 Future<List<Map>> getData();
 Future<bool> insertItem(Map<String, dynamic> item);
 Future<bool> deleteItem(int index);
 Future<bool> updateItem(int index, Map<String, dynamic> item);
 static AbstractTodoRepo? _todoInstance;

 static AbstractTodoRepo getInstance() {
   _todoInstance ??= TodoDB();
   return _todoInstance!; // For backend data
 }
}
```

Data Persistence using Flutter

05

- **Steps to started : Optional ! Use more OOP**
  - Create a folder **models** inside **lib**
    - Create a Model for each table
      - Define the attribute as instance variable
      - Create the converter functions to Map (and even statically from a Map)

Data Persistence using Flutter

05

```
class Todo {
 final int id;
 final String title;
 final bool done;

 const Todo({
   required this.id,
   required this.title,
   required this.done,
 });
 Map<String, dynamic> toMap() {
   return {
     'id': id,
     'title': title,
     'done': done,
   };
 }
 @override
 String toString() {
   return 'Todo{id: $id, title: $title, done: $done}';
 }
}
```

Data Persistence using Flutter

05

- **Steps to started : Optional ! Use more OOP**
  - Integrate the Model Todo class into the database helper functions

```
const SizedBox(width: 10),
ElevatedButton(
  onPressed: () {
    _tx_title_controller.text = '';
    TodoDB.insertToDo({'title': _tx_title_value, 'done': 0});
    setState(() {});
  },
  child: const Text(
    "Add",
  ),
```

Data Persistence using Flutter

05

- **Steps to started : Optional ! Use more OOP**
  - Integrate the Model Todo class into the database helper functions

```
const SizedBox(width: 10),
ElevatedButton(
  onPressed: () {
    _tx_title_con
    TodoDB.insert
    setState(() {
  },
  child: const Te
    "Add",
  ),
```

```
const SizedBox(width: 10),
ElevatedButton(
  onPressed: () {
    _tx_title_controller.text = '';
    var myTodo=Todo(id:0 , title:_tx_title_value,done: false)
    TodoDB.insertToDo(myTodo);
    setState(() {});
  },
  child: const Text(
    "Add",
  ),
```

Data Persistence using Flutter

05

- **Steps to started : Optional ! Use more OOP**
  - Integrate the Model Todo class into the database helper functions

```dart
static Future insertToDo(Map<String, dynamic> data) async {
  final database = await DBHelper.getDatabase();
  database.insert("todo", data, conflictAlgorithm: ConflictAlgorithm.replace);
}
```

```dart
static Future insertToDo(Todo todo) async {
  final database = await DBHelper.getDatabase();
  database.insert("todo", todo.toMap() , conflictAlgorithm: ConflictAlgorithm.replace);
}
```

Section 5

Questions

Question 1 : Pre-Load List of Countries and Regions

- ***We have a large list of countries and regions/cities, how to add them to the mobile app so that the user would choose their address (Country + City) :***

Question 1 : Pre-Load List of Countries and Regions

- ***We have a large list of countries and regions/cities, how to add them to the mobile app so that the user would choose their address (Country + City) :***
  - *Hardcode them ?*
  - *Load them from the Web ?*
  - *Pre-populate the data in SQLite ?*
  - *Use JSON data as an Asset ?*

Question 2 : Data Synching between Apps and Services

- **Concepts and Techniques**
  - **Offline Use :**
    - Most mobile apps need to have a local database for the case of offline use :
      - Users can save data, add photos…
        - Data is saved locally in SQLite
        - When there is an internet connection, data are uploaded to the backend or other external services.

Question 2 : Data Synching between Apps and Services

- **Concepts and Techniques**
  - **Implement Caching Techniques :**
    - Images or large static data, store them at a local cache (Database, file system) in the same way as browsers to reduce the number of future connections/bandwidth.

Question 2 : Data Synching between Apps and Services

- **Concepts and Techniques**
  - **Sync Always in the background to the Backend Servers :**
    - Either doing full-sync or incremental Sync, Try to do it in the background using Cron (Or other plugins) :
  - There are cases where you may sync on the user's requests

```
Future<void> main() async {
 ...
 final cron = Cron();
 cron.schedule(Schedule.parse('*/5 * * * *'), () async {
   actionUploadImageUpload();
 });
```

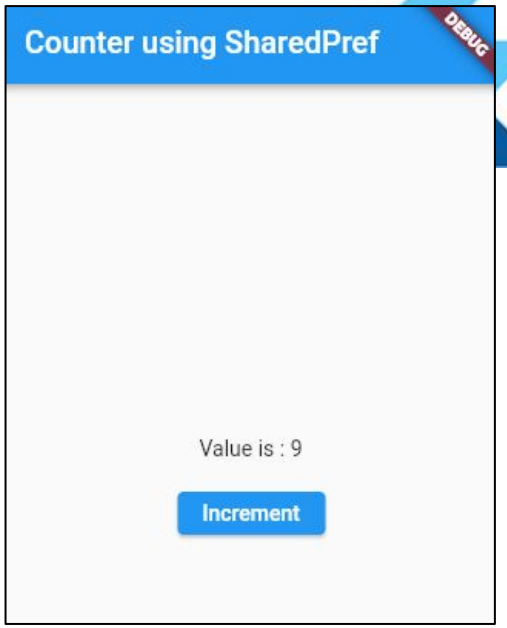
Question 2 : Data Synching between Apps and Services

- **Concepts and Techniques**
  - **Sync from backend to Mobile Apps**
    - Use Push Notifications so that the backend servers notify subscribed apps when there is a change or a trigger
    - Excessive calls from mobile apps to the backend can be expensive in terms of cost + battery + server performance.

Question 3 : Multi-Device Increment App

Mobile A

How to keep the incremented value **synched** across different devices ?



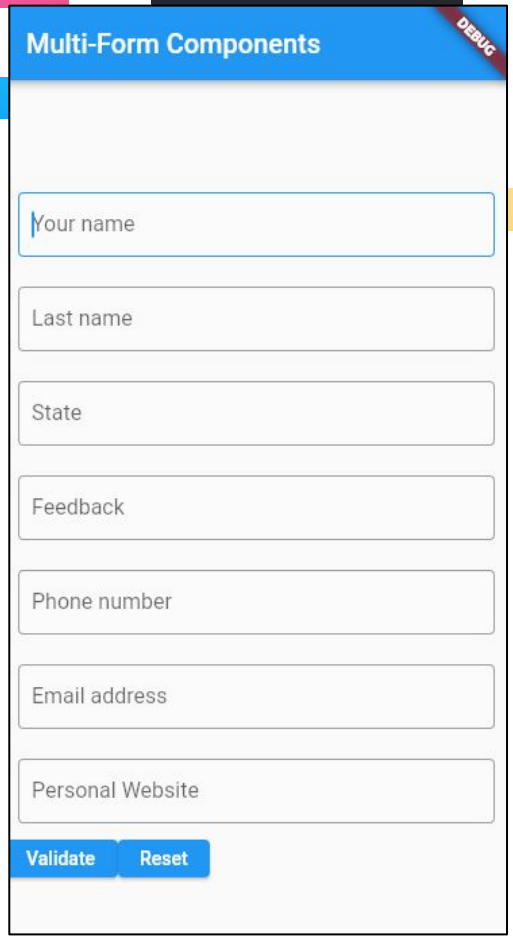
Mobile B

Resources

- https://docs.flutter.dev/cookbook/persistence/key-value
- https://docs.flutter.dev/cookbook/persistence/sqlite

Questions from Students

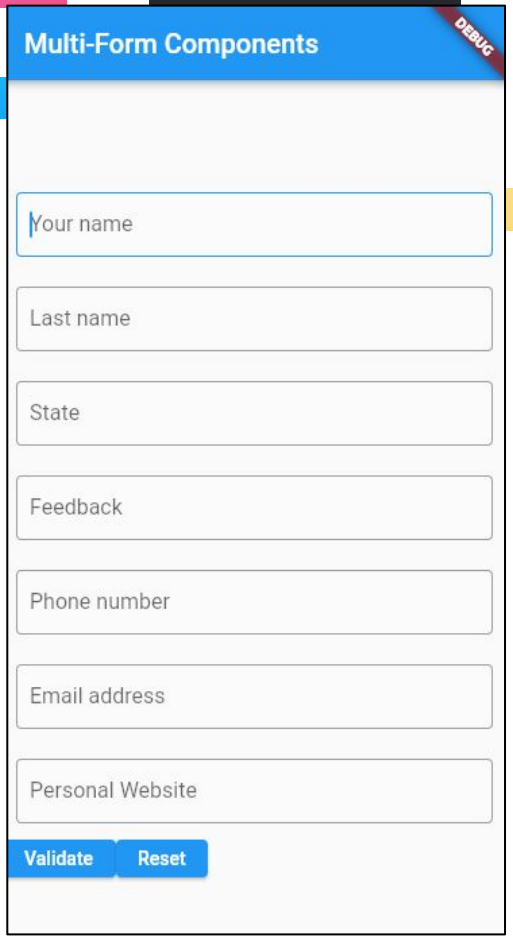
- **A Flutter Screen contains many text fields that we need to individually for each text input:**
  - Show Hint Text
  - Track the value typed by the user
  - Its own validation code(s)
- What's the optimal way to write the flutter code without copying and pasting. (Respecting DRY)



Asked by Anis Rahmani

form_data.dart

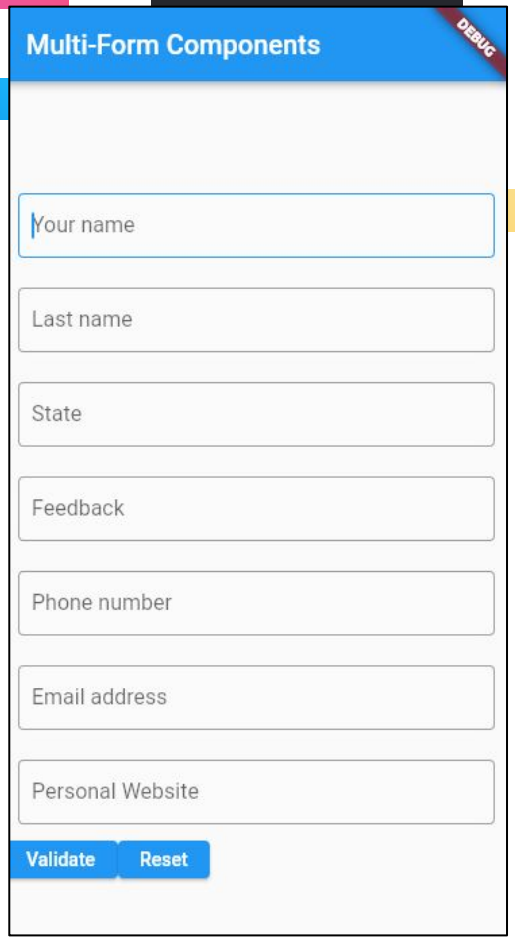
```dart
import 'package:flutter/material.dart';

Map form_items = {
 'name': {
   'hint': 'Your name',
   'value': '',
   'controller': TextEditingController(),
   'validation': ['verify_length_over5', 'is_not_empty'],
 },
 'lastname': {
   'hint': 'Last name',
   'value': '',
   'controller': TextEditingController()
 },
 'state': {
   'hint': 'State',
   'value': '',
   'controller': TextEditingController()
 },
 'feedback': {
   'hint': 'Feedback',
   'value': '',
   'controller': TextEditingController()
 },
 'phone': {
   'hint': 'Phone number',
   'value': '',
   'controller': TextEditingController()
 },
 'email': {
   'hint': 'Email address',
   'value': '',
   'controller': TextEditingController()
 },
 'Website': {
   'hint': 'Personal Website',
   'value': ''
```



Asked by Anis Rahmani

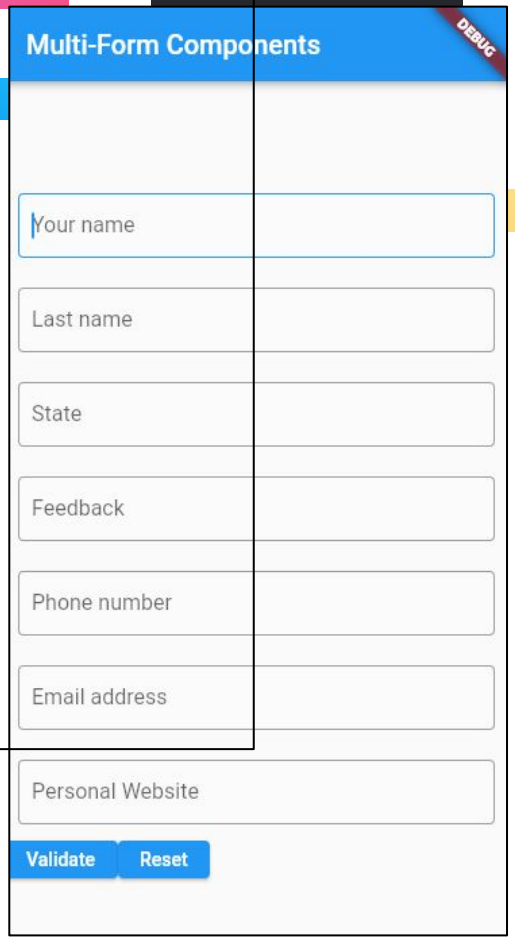
validation.dart

```dart
//Shame we don't have Eval on Dart.

String? eval_validation_string(String name,
String value) {
 if (name == 'verify_length_over5') return
verify_length_over5(value);
 if (name == 'is_not_empty') return
is_not_empty(value);
 return null;
}

String? verify_length_over5(String value) {
 if (value.length <= 5) {
   return 'Value must be > 5';
 }
 return null;
}

String? is_not_empty(String value) {
 if (value.isEmpty) {
   return 'Field must be left empty';
 }
 return null;
}
```



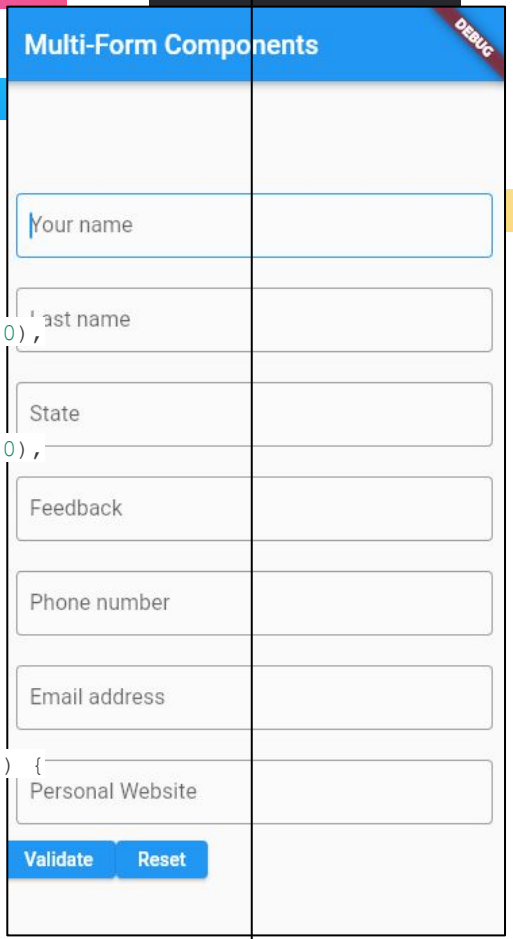
homescreen.dart

```dart
class _HomeScreenState extends State<HomeScreen> {
 final _formKey = GlobalKey<FormState>();

 @override
 Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(title: Text("Multi-Form Components")),
     body: Form(
       key: _formKey,
       child: Column(
         mainAxisAlignment: MainAxisAlignment.center,
         crossAxisAlignment: CrossAxisAlignment.center,
         children: [



           ],
         )
       ],
     ),
   ),
 );
 }
}
```



homescreen.dart

```
children: [
  for (var k in form_items.keys)
    Container(
      height: 50.0,
      margin: EdgeInsets.all(10),
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(13.0),
        color: Colors.white24,
      ),
      child: TextFormField(
        decoration: InputDecoration(
          focusedBorder: OutlineInputBorder(
            borderSide:
                const BorderSide(color: Colors.blue, width: 1.0),
          ),
          enabledBorder: const OutlineInputBorder(
            borderSide:
                const BorderSide(color: Colors.grey, width: 1.0),
          ),
          hintText: form_items[k]!['hint'],
          contentPadding: EdgeInsets.only(left: 10.0),
        ),
        onChanged: (value) {
          form_items[k]!['value'] = value;
        },
        controller: form_items[k]!['controller'],
        validator: (value) {
          if (form_items[k]!['validation'] is List) {
            for (var validFunc in form_items[k]!['validation']) {
              var ret = eval_validation_string(
                  validFunc, form_items[k]!['value']);
              if (ret != null) return ret;
            }
          }
        },
      ),
```



Asked by Anis Rahmani

homescreen.dart

```
Row(
  children: [
    ElevatedButton(
        onPressed: () {
          if ( formKey.currentState!.validate()) {
            ScaffoldMessenger.of(context).showSnackBar(
              const SnackBar(content: Text('Processing Data')),
            );
          }
        },
        child: Text("Validate")),
    SizedBox(
      width: 20,
    ),
    ElevatedButton(
        onPressed: () {
          for (var k in form_items.keys) {
            (form_items[k]!['controller'] as TextEditingController)
                .text = '';
          }
        },
        child: Text("Reset"))
  ],
```